

Programming of distributed systems

Exercise 1: Remote objects

In this first practice a program with the following functionality is provided:

File Manager

This program implements several functions to manage the files in a local directory called "FileManagerDir", and performs basic read/write functions on them. The program is composed of the following files:

- **fileManager.h**: This class implements the functions to manage files of a local directory to the executable containing this class. The name of the local folder is passed by parameters to the constructor. The class has the following functions:
 - **ListFiles**: Returns a `std::vector` with the names of the files found in the local directory. WARNING, it is not a dynamic list, it is not updated automatically if the files are modified. If new files are written/created they will not be added automatically, the function must be invoked again to obtain the updated list.
 - **ReadFile**: This function reads the contents of a file in the local folder. Given the name of one of the files found in the directory (list returned by "ListFiles"), it will read its size and content. It will store it in the variables passed by parameters.
 - **WriteFile**: Analogous to the previous one, it will write the file content in the local directory. If the file did not exist in the directory, it will be added to the list of files (you have to call "ListFiles" again if you want to keep the list updated).
- **LibFileManager.a**: This file contains the implementation of the class mentioned above. It is already compiled and the project "CMakeLists.txt" is prepared to link it to the executable.
- **Main_fm.cpp**: Source code for the main test program. It creates a class of type "fileManager" and provides a command line terminal for typing commands. You have the following commands:
 - **ls**: list files local to the program.
 - **lls**: lists the files local to the "fileManager" class. They will be the files that are in the directory passed by parameters to the constructor of that class.
 - **upload**: This command copies a file local to the executable to the directory local to the fileManager class (the one passed by parameters to the constructor). This command receives one additional parameter:
 - Name of the "source" file, which is located in the directory local to the executable.
 - **EXAMPLE**:
 - Executing the following command will copy the file "a.txt" to the directory local to fileManager.
 - "upload a.txt"

- **exit():** This command closes the terminal and terminates the program execution.
- **download:** This command copies a file local to the fileManager class (the directory passed by parameters to the constructor) to the local directory of the executable. This command receives one additional parameter:
 - Name of the “source” file, which is located in the directory local to fileManager.
 - EXAMPLE:
 - Executing the following command will copy the file “a.txt” to the directory local to the executable.
 - “download a.txt”

Goals:

The fileManager class must be distributed in such a way that it is as transparent as possible to the supplied “main” program. The files supplied with this practice (fileManager.h and main_fm.cpp) cannot be modified. The student must implement at least two programs, that can be on the same computer using “localhost” and different ports:

- Program 1 (server): will act as an object server of type “fileManager”, providing the remote implementation of the previous classes.
- Program 2 (client): It will act as a client, using the code from “main_fm.cpp”. This program will make the necessary calls to the remote classes contained in the server of program 1.

Extra exercise:

Create an “Object Broker” program to find services of a given type and balance connections:

- FileManager server type programs may be located on an unknown computer, so the client would need a service finder system. Also, in order to balance connections between multiple computers, there may be multiple instances of the server program on the system.
 - In order to solve these problems, it is requested to create a Server type program that will work as an “Object Broker”, which will have the following operation:
 - It will wait for fileManager client or server programs to connect:
 - If a server is connected, it must be identified with its IP and the Broker will store it.
 - If a client connects, a stored IP of one of the previously connected servers will be returned and that connection will be noted. The Broker will try to return the IP of the server with the least number of connections.
 - The client/server programs will work as follows:
 - Server program: Initiates a connection to the broker and sends its private network IP. Then waits for normal client connections.

- Client program: Before initiating a connection to the server, it will connect to the Broker to request an IP of any server that is active.
- In order to simplify the implementations, both client/server programs can know both their own IP and that of the Broker. The client only knows the server IP if it is given by the Broker and vice versa.

In order to test this part, the students should create two more EC2 machines (three in total). Two for the fileManager server and one for the Broker program. The client can be launched from any of those machines.

Submission

The practices are individual, the students must submit the code developed through blackboard, and take an exam in which the final grade of the exercise will be obtained.

The practices must be delivered before the day indicated in its delivery section in Blackboard. In case of delivering it after the deadline, there is a risk of not being able to take the exam of the practice. The content delivered in Blackboard will have the following structure:

- ZIP file with the files generated/edited by the student. Each of the files delivered MUST contain the name of the student written in code comments at the beginning of each file. The student is free to choose the directory structure of the file.
- File name (REQUIRED): PR1SISDIS_StudentName_Surname.zip

Any file submitted that does not comply with these requirements may be ignored. Please do not submit files with other compressors or nomenclatures.